

# Sprog og oversættere

Lexical analysis: Regular expressions and NFAs

Part 1: Learning goals: Some basic definitions

Hans Hüttel

(using content by Cosmin Oancea)

Creative Commons License  
Attribution Non Commercial 4.0 International  
(CC-BY-NC-4.0)



# Learning goals

- To be able to explain the purpose of lexical analysis in the context of interpreters and compilers
- To be able to use and apply the basic terminology for formal languages
- To be able to describe the form of regular expressions and what they can describe
- To be able to use regular expressions to describe lexical units of programming languages
- To be able to explain the definition of a nondeterministic finite automation and what it means for an NFA to accept a string
- To be able to explain and use the algorithm for converting a regular expression to an NFA and explain how this is related to lexical analysis

# Please note!



If you follow the course **Syntax and semantics**, many of the notions that follow also appear in that course.

But the notation is a little different, and the focus is of course a bit different as well.

The following notions are very important in computer science, so one has to become familiar with them and be able to use the terminology correctly.

Please pay close attention!

# Strings and languages

## Definition (Alphabet)

An **alphabet**  $\Sigma$  is a finite set of symbols that we call **characters**.

## Definition (Strings and languages)

Let  $\Sigma$  be an alphabet.

- **A string** over  $\Sigma$  is a finite sequence of characters  $w = a_1 a_2 \dots a_n$ ,  $a_i \in \Sigma$ . If  $n = 0$  we get the empty string, denoted  $\varepsilon$ .  $\Sigma^*$  is the set of all strings over  $\Sigma$ .
- **A language**  $L$  over  $\Sigma$  is a set of strings over  $\Sigma$ , i.e.,  $L \subseteq \Sigma^*$ .

---

<sup>1</sup>Dansk: *mængde*



# Strings and languages

## Definition (Alphabet)

An **alphabet**  $\Sigma$  is a finite set of symbols that we call **characters**.

## Definition (Strings and languages)

Let  $\Sigma$  be an alphabet.

- **A string** over  $\Sigma$  is a finite sequence of characters  $w = a_1 a_2 \dots a_n$ ,  $a_i \in \Sigma$ . If  $n = 0$  we get the empty string, denoted  $\varepsilon$ .  $\Sigma^*$  is the set of all strings over  $\Sigma$ .
- **A language**  $L$  over  $\Sigma$  is a set of strings over  $\Sigma$ , i.e.,  $L \subseteq \Sigma^*$ .

Please remember: There is no notion of "meaning" involved in these definitions! A language is simply a set<sup>1</sup>, so we can use set notation in the usual way.

<sup>1</sup>Dansk: *mængde*

# Some useful notation

## Definition

If  $u = a_1 \dots a_m$  and  $v = b_1 \dots b_n$  are strings, the **concatenation** of  $u$  and  $v$  is

$$uv = a_1 \dots a_m b_1 \dots b_n$$

## Definition

If  $k \geq 1$  and  $u$  is a string, then

$$u^k = \underbrace{u}_{k \text{ times}}$$

# Some examples of languages

Let  $\Sigma$  be the alphabet of lowercase letters in the Latin alphabet.

- The total language  $\Sigma^*$  and the empty language  $\emptyset$
- The language of C keywords: {if, else, return, do, while, for, ...}
- $\{a^n b^m \mid n, m \geq 1\} = \{ab, aab, aaab, \dots, \dots aabbb \dots\}$
- $\{a^n b^n \mid n \geq 0\} = \{\epsilon, ab, aabb, \dots\}$
- The language of palindromes:  
{deified, madam, regnirger, pip, Mads Dam, ...}